

ILCMS Technical Specification v2.1

AI Implementation & RAG Engineering Specification

Icelandic Legal Case Management System (ILCMS)

Developer Implementation Baseline

3 September 2026

Implementation Position

- v2.1 is the build-oriented companion to v2.0 and defines concrete services, schemas, pipelines, interfaces, security controls and deployment.
- Initial RAG storage is PostgreSQL + pgvector; local inference is a first-class target; AI never bypasses Version 1 authorization, records, workflow or audit.

1. Purpose

- v2.1 converts v2.0 into an implementation-level engineering specification. It defines services, database structures, the RAG pipeline, APIs, workers, model interfaces, security boundaries, Kubernetes deployment, testing and operations.
- The design assumes the Version 1 platform and preserves its authorization, records-management, workflow and audit boundaries.

2. Implementation Principles

- Authoritative data remains in PostgreSQL and object storage.
- AI is optional; disabling AI must not disable normal case management.
- Authorization is enforced before retrieval and before source presentation.
- Retrieved documents are untrusted data, never executable instructions.
- Model, embedding model, prompt, policy, retrieval configuration and evaluation versions are recorded.
- High-impact outputs require explicit human approval; retryable workers are idempotent.

3. Technology Baseline

- Python 3.12+, FastAPI, Pydantic v2, SQLAlchemy 2.x, Alembic.
- PostgreSQL + pgvector initially; PostgreSQL FTS/pg_trgm plus vector retrieval for hybrid search.
- S3-compatible object storage, RabbitMQ, Redis, Vault/External Secrets.
- Local inference through vLLM or equivalent OpenAI-compatible server; Kubernetes GPU node pool.
- OpenTelemetry, Prometheus, Grafana and structured privacy-aware logging.

4. AI Services

- ai-orchestrator coordinates requests and persists run state.
- ai-policy-service evaluates use-case, user, organisation, data class, model and provider policy.
- ai-retrieval-service performs authorization-aware hybrid retrieval.
- ai-model-gateway provides provider-neutral LLM/embedding interfaces.
- ai-document-worker performs OCR, extraction, chunking and indexing.
- ai-evaluation-service runs regression suites; ai-admin manages models, prompts, policies and incidents.

5. Repository Structure

- backend/app/ai/domain; backend/app/ai/application; backend/app/ai/infrastructure/models; backend/app/ai/infrastructure/providers; backend/app/ai/infrastructure/retrieval; backend/app/ai/api.
- worker/app/jobs/ai; evaluation/; deploy/helm/ai; prompts/.

6. Core AI Database Schema

- ai_providers(id, name, type, endpoint, data_region, retention_policy, training_policy, enabled).
- ai_models(id, provider_id, name, version, digest, capability_json, context_limit, licence, deployment_type, status, published_at, retired_at).
- ai_prompts(id, key, version, language, template, input_schema, output_schema, status, owner_user_id, published_at).
- ai_policies(id, key, version, risk_class, allowed_data_classes, allowed_models, external_allowed, human_approval_required, max_context_tokens, retention_days, status).
- ai_runs(id, organization_id, case_id, user_id, use_case, risk_class, status, model_id, prompt_id, policy_id, retrieval_config_json, request_id, started_at, completed_at, metrics_json, error_code).

- `ai_run_sources(run_id, document_version_id, chunk_id, rank, retrieval_method, retrieval_score, citation_key)`. `ai_run_outputs(id, run_id, output_type, content, structured_json, validation_status, approval_status, approved_by, approved_at)`.

7. RAG Storage

- `document_chunks(id, document_version_id, case_id, organization_id, chunk_index, content, content_hash, page_start, page_end, section_path, token_count, language, classification, acl_snapshot_hash, created_at)`.
- `document_embeddings(id, chunk_id, embedding_model_id, dimensions, embedding, content_hash, created_at)`, unique on chunk/model.
- Every chunk carries access-control metadata. OWASP RAG guidance recommends propagating access controls to chunks and enforcing filtering before retrieval rather than retrieving restricted material and filtering afterwards.

8. Document-to-RAG Pipeline

- Upload → validation → malware scan → authoritative document version → OCR → deterministic metadata → structural extraction → chunking → content hash → ACL/classification metadata → embeddings → lexical/vector index → READY → audit/outbox event.

9. Chunking Algorithm

- Prefer headings, paragraphs, numbered clauses and logical table groups. Preserve page and section metadata. Do not split legal numbering from its clause.
- Initial target: approximately 300–700 tokens with 50–100 token overlap; tune against an Icelandic legal evaluation corpus.

10. Query/Retrieval Algorithm

- Normalize Unicode/whitespace while retaining the original query. Determine case/document scope and structured filters.
- Authorize the request and build ACL filters. Run lexical and vector searches independently. Start with 20–50 candidates per method. Fuse rankings using Reciprocal Rank Fusion; optionally rerank. Apply context budget, source diversity and final authorization checks.

11. Retrieval Security

- Authorization filtering is a hard security boundary. Do not expose raw similarity scores. Rate-limit retrieval to reduce corpus probing. Re-check authorization when opening/exporting sources. Index writers are limited to controlled ingestion pipelines.

12. Prompt Construction

- Separate system policy, user request, retrieved data and output requirements. Delimit retrieved content as untrusted data. Never let document text override system policy. Include stable source identifiers. Use JSON Schema for structured extraction. OWASP identifies prompt injection as a major LLM/RAG risk and recommends least privilege, untrusted-content segregation, filtering and human approval.

13. Context and Citations

- Use a policy-configurable context budget, initially around 4,000–12,000 tokens depending on use case/model. Prefer diverse authoritative sources and the latest authorized version unless historical material is requested.
- Required citation keys must map to `ai_run_sources`. Fabricated citation IDs are rejected/downgraded. Citations identify document/version and page/section where available and open only through normal authorization.

14. LLM Provider Interface

- Internal interface: `generate()`, `generate_structured()`, `embed()`, `health()`, `capabilities()`. Provider adapters translate to provider-specific APIs and contain no business authorization logic.
- External providers are disabled unless explicitly permitted. Local providers use service authentication and `NetworkPolicies`.

15. Model and Prompt Routing

- Users select a use case, not arbitrary production models. Policy chooses an approved model class. Extraction may use a smaller structured model; complex analysis may use a larger approved model.
- Published model/prompt versions are immutable. Every run records actual versions.

16. RAG Index Generations

- Track embedding model, dimensions, chunking version and creation time in an `index_generation` record. States: BUILDING, VALIDATING, ACTIVE, RETIRED, FAILED.

- Changing embedding models creates a new generation; validate retrieval quality before activation. Source hashes permit incremental re-embedding.

17. AI Run State Machine

- QUEUED → AUTHORIZING → RETRIEVING → GENERATING → VALIDATING → AWAITING_APPROVAL or COMPLETED. Failure: REJECTED, FAILED, CANCELLED.
- Approval: AWAITING_APPROVAL → APPROVED or REJECTED. No high-impact run may directly create an external side effect.

18. API Contracts

- POST /api/v1/ai/runs; GET /api/v1/ai/runs/{id}; POST /api/v1/ai/chat; POST /api/v1/ai/summarize; POST /api/v1/ai/extract; POST /api/v1/ai/research; POST /api/v1/ai/deadline-suggestions; POST /api/v1/ai/draft; GET /api/v1/ai/sources/{id}; POST /api/v1/ai/runs/{id}/approve.
- Long-running operations return HTTP 202 with run_id. Responses include provenance and citations as applicable.

19. Asynchronous Jobs

- ai.document.index; ai.embedding.generate; ai.rag.query; ai.summary.generate; ai.extract.generate; ai.deadline.detect; ai.evaluation.run; ai.index.rebuild; ai.model.healthcheck.
- Use retry limits, exponential backoff, dead-letter handling and stable job/run IDs.

20. Human Approval

- Approval UI shows output, citations, warnings, model/prompt/policy versions and proposed side effects. Actions: ACCEPT, ACCEPT_WITH_EDITS, REJECT, ESCALATE.
- Approved content enters normal ILCMS document/workflow lifecycle. Approval records the human decision; it does not imply model correctness.

21. AI Deadline Intelligence

- Extract candidate date/event/obligation/source. Store as DEADLINE_PROPOSAL with document/page/text/model provenance. Reviewer confirms/rejects/edits. Only confirmed deadlines use the normal deadline service. Deterministic deadline rules remain authoritative.

22. Document Classification

- AI may propose document type, parties, case relation, confidentiality and classification. Keep AI extraction separate from approved authoritative metadata. High-impact classification requires review.

23. Air-Gapped Implementation

- Run model server, embedding server, OCR, retrieval, orchestrator and evaluation services inside private Kubernetes. Mirror and verify container/model artefacts in internal repositories before disconnection. Use internal TLS/DNS/Vault and disable external egress at network level.
- Provide an offline model-import and integrity-verification process.

24. vLLM/Kubernetes Deployment

- Use an internal ClusterIP for the model server, GPU-labelled nodes, taints/tolerations, resource limits, PDBs and NetworkPolicies. vLLM offers OpenAI-compatible APIs and a Kubernetes production stack.
- Do not rely on vLLM API-key authentication alone: its documentation notes that not every endpoint is protected. Place it behind the platform's network/security boundary.

25. Security Threat Model

- Test prompt/indirect injection, cross-case leakage, malicious PDFs, RAG poisoning, model supply-chain compromise, external-provider leakage, excessive tool privileges, denial-of-service, fabricated citations and false legal claims.
- Retrieved content must always be treated as untrusted. AI tools are allowlisted and least-privileged.

26. Agent/Tool Model

- v2.1 does not require autonomous agents. Tools are application functions with schema, permission, risk class and audit event. No unrestricted shell, SQL, filesystem or network tool is exposed.

- High-risk write/external tools require explicit confirmation immediately before execution.

27. Evaluation Dataset

- Build a versioned Icelandic legal-domain dataset covering cases, contracts, pleadings, decisions and correspondence, in Icelandic and English. Include ambiguity, missing evidence, conflicting sources and adversarial documents.
- Prefer synthetic/de-identified data unless governance explicitly permits real confidential material.

28. Evaluation Metrics and Gates

- Retrieval: Recall@K, Precision@K, MRR/NDCG. Generation: citation correctness, groundedness, factual accuracy, relevance and refusal quality. Extraction: precision/recall/F1. Operations: p50/p95 latency, failure rate, queue time, tokens and GPU use.
- Security leakage must be zero in regression tests. High-risk citation fabrication must be zero. Model/prompt/index changes require evaluation before publication.

29. Observability

- Correlate request_id → ai_run_id → policy → retrieval → model → validation → approval. Metrics include runs, failures, latency, retrieval latency, policy denials, citation failures, external calls, tokens and GPU utilisation.
- Never log full confidential prompts/documents by default.

30. Data Retention

- Retention is policy-driven by use case/data class. Account for runs, prompts, outputs, embeddings, indexes, caches and evaluation copies. Legal holds override normal destruction where applicable.

31. CI/CD

- Run static analysis, unit/integration tests, tenant-isolation tests, RAG regression tests, prompt-injection tests, container/SBOM scanning, model digest verification and offline build tests. Model/prompt/index publication requires evaluation evidence.

32. Implementation Roadmap

- Sprint 1: schema, interfaces, model/prompt/policy registries and feature flags.
- Sprint 2: chunking and pgvector index generation.
- Sprint 3: authorization-aware hybrid retrieval.
- Sprint 4: local model adapter and RAG chat.
- Sprint 5: citations, validation and audit.
- Sprint 6: summarisation/extraction.
- Sprint 7: deadline intelligence/classification.
- Sprint 8: evaluation, red-team tests and production hardening.

33. Initial Developer Backlog

- AI-101 AI schema/migrations; AI-102 provider/model adapter; AI-103 prompt registry; AI-104 policy evaluator; AI-105 chunking; AI-106 pgvector generation; AI-107 hybrid retriever; AI-108 ACL retrieval tests; AI-109 vLLM adapter; AI-110 citation validator; AI-111 RAG endpoint; AI-112 audit trail; AI-113 approval API/UI; AI-114 deadline pipeline; AI-115 evaluation harness; AI-116 injection tests; AI-117 GPU deployment; AI-118 observability dashboards.

34. Acceptance Criteria

- Local deployment works without external AI connectivity. Documents can be ingested, chunked, embedded and indexed. Authorized users receive grounded case-scoped answers with citations.
- Unauthorized semantic queries cannot retrieve restricted chunks. Permission changes affect subsequent retrieval. Runs retain model/prompt/policy/source provenance. AI deadlines remain proposals until confirmation. External AI can be disabled centrally. Core business logic is independent of models.

35. Engineering References

- pgvector documentation covers exact/approximate vector search, HNSW/IVFFlat, filtering and hybrid search. vLLM documents OpenAI-compatible serving and Kubernetes deployment. OWASP documents prompt-injection and RAG security controls. NIST AI RMF's Generative AI Profile provides lifecycle-oriented risk-management and evaluation guidance.

Appendix A — Reference Architecture Flow

- Document Service → OCR/Extraction → Structural Chunking → Embedding → PostgreSQL FTS/pgvector
- User → ILCMS API → AI Orchestrator → Policy/Authorization → Hybrid Retrieval → Context Builder → Model Gateway → Validation → Human Review → ILCMS

Appendix B — Non-Negotiable Rules

1. Authorization is evaluated before retrieval.
2. Retrieved content is untrusted data.
3. AI cannot grant itself permissions.
4. AI cannot directly modify authoritative legal records.
5. External providers require explicit policy approval.
6. Every production run records model/prompt/policy/source provenance.
7. Vector indexes are derived and rebuildable.
8. High-impact actions require human approval.
9. Published model and prompt versions are immutable.
10. AI can be disabled without breaking core ILCMS.

Appendix C — First Demonstrable Milestone

Build a fully local case-scoped RAG assistant: Icelandic legal PDF → OCR → structural chunking → embedding → authorization-aware hybrid retrieval → local LLM → page-level citations → auditable AI run. This validates the core security and architecture before broader legal-intelligence features.